

# Dispensa Weekly Status Report

**28 September - 2 October 2026**

Prepared by Aref Haghgoorostami

dispensa-api - stock-management backend for small restaurants

# Executive summary

## AHEAD

This week's code already covers roadmap weeks 1-3 (design doc, project skeleton, database, data models and a full CRUD feature) three calendar weeks into a ten-week plan.

- Wrote and self-reviewed docs/DESIGN.md v1 before any code existed, catching a real data-integrity bug (signed stock quantities vs. an overly strict CHECK constraint).
- Stood up the FastAPI project skeleton, a pre-commit quality gate (ruff + mypy), and PostgreSQL 16 running in Docker Compose.
- Modeled restaurants, users and ingredients in typed SQLAlchemy 2 and shipped the first Alembic migration.
- Delivered the first end-to-end feature: ingredients CRUD with tenant scoping, a consistent error envelope, and 11 automated tests - including a restaurant-isolation test.
- Started docs/WORKLOG.md as a running log of decisions and lessons learned each session.

## Numbers at a glance

|                        |                            |                                                     |                           |
|------------------------|----------------------------|-----------------------------------------------------|---------------------------|
| <b>7</b><br>PRs merged | <b>11</b><br>Tests passing | <b>2%</b><br>Roadmap checkboxes<br>ticked (1 of 58) | <b>0</b><br>Open blockers |
|------------------------|----------------------------|-----------------------------------------------------|---------------------------|

The roadmap-progress card counts only the small checkboxes ticked inside ROADMAP.md itself, not the actual amount of work done - see the note on the next page.

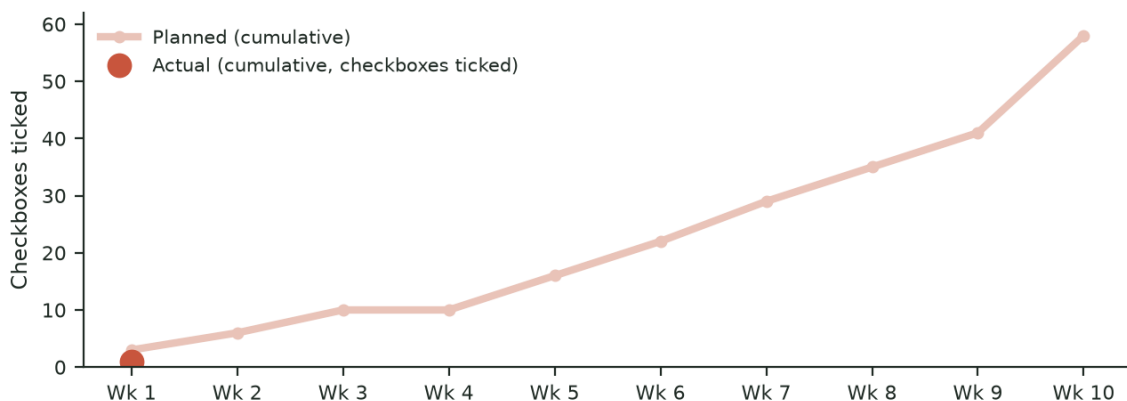
## Progress vs. plan

This week's roadmap items - from weeks 1 to 3 of ROADMAP.md, all of which were touched this week - and their real status, independent of whether the checkbox in the file was ticked.

| Planned                                                                     | Status      | Notes                                               |
|-----------------------------------------------------------------------------|-------------|-----------------------------------------------------|
| GitHub profile: photo, bio, profile README                                  | Not started | Carried over to next week                           |
| Create dispensa-api repo (README, ROADMAP, CLAUDE.md)                       | Done        | PR scaffold + initial commit, 28 Sep                |
| docs/DESIGN.md sections 1-4 (requirements, architecture, API, auth/tenancy) | Done        | PR #4, 29 Sep                                       |
| docs/DESIGN.md sections 5-9 (data model, consistency, perf, jobs, ops)      | Done        | Also PR #4 - written as one v1 pass                 |
| Design review with Claude Code; fix what it finds                           | Done        | 1 real bug + 4 ambiguities found and resolved       |
| Project skeleton: FastAPI, Ruff, mypy, pre-commit, /health running          | Done        | PR #5, #6                                           |
| PostgreSQL in Docker Compose                                                | Done        | PR #7                                               |
| SQLAlchemy models: restaurants, ingredients                                 | Done        | PR #8 - users model added too                       |
| First Alembic migration                                                     | Done        | PR #8, reviewed by hand, upgrade/downgrade verified |
| Ingredients CRUD (create, list, read, update, delete)                       | Done        | PR #9 - archive instead of delete, 11 tests         |

Net effect: everything planned through roadmap week 3 is functionally built. What's missing is bookkeeping - ROADMAP.md's own checkboxes for weeks 1-3 were mostly left unticked as the work landed (57 of 58 across the whole roadmap are still unchecked). Fixing that is the first item in "Next week" below.

## Roadmap progress: planned vs. actual



This is the project's first week, so there is only one week of test-count history - a chart of tests over time will start appearing from next week's report.

## Delivered this week

One block per pull request merged this week, in the order it happened. No GitHub Actions CI exists yet (planned for roadmap week 5), so "checks run" below are the checks actually run by hand before each merge.

### #4 - docs: add system design v1

What: Produced the one reference document the whole project is built against, and used it to catch design mistakes before they became code.

How:

- Wrote docs/DESIGN.md v1: requirements, architecture, API design, auth/tenancy, data model, the stock-ledger pattern, performance, background jobs and operations
- Self-review found and fixed a real bug: stock\_movements.quantity was signed, but the original CHECK only allowed positive values
- Resolved four ambiguities: correction semantics, report-month timezone, availability flooring, waste vs. negative stock
- Synced ROADMAP.md's endpoint table and moved recipes/sales into the MVP scope

Evidence: No automated tests (a design doc); reviewed by hand, point by point, before merging.

<https://github.com/Arefhg/dispensa-api/pull/4>

### #5 - feat: add project skeleton with FastAPI /health endpoint

What: Got the first version of the program actually running and answering a request over HTTP.

How:

- pyproject.toml: FastAPI, uvicorn, pytest, ruff, mypy dependencies and tool config
- Python 3.12 venv pinned via .python-version
- app/main.py: FastAPI app with GET /health -> {"status": "ok"}
- README rewritten with real setup/run instructions

Evidence: pytest: 1 passed. ruff check, ruff format --check and mypy --strict all clean. Manually verified /health, /docs and /openapi.json each return 200.

<https://github.com/Arefhg/dispensa-api/pull/5>

### #6 - chore: add pre-commit (ruff check, ruff format, mypy)

What: Made code-quality problems get caught before a commit exists, not minutes later in CI.

How:

- .pre-commit-config.yaml: ruff-check, ruff-format and mypy hooks, with FastAPI/httpx types added to mypy's isolated environment
- Installed the git hook and documented the step in README

Evidence: pre-commit run --all-files passed. Proved the gate actually blocks bad commits by deliberately committing an unused import and watching it get refused.

<https://github.com/Arefhg/dispensa-api/pull/6>

## #7 - feat: add PostgreSQL via Docker Compose and database connection

What: Gave the app a real, persistent database instead of nothing to store data in.

How:

- docker-compose.yml: PostgreSQL 16, named volume, healthcheck, port bound to 127.0.0.1 only
- app/config.py: pydantic-settings Settings reading DATABASE\_URL from the environment
- app/db.py: sync SQLAlchemy 2 engine/session with a constraint-naming convention
- Alembic initialised, wired to app.config instead of a duplicated URL

Evidence: pytest: 1 passed. Manually: docker compose up, confirmed healthy, connected via both psql and the app's own engine.

<https://github.com/Arefhg/dispensa-api/pull/7>

## #8 - feat: add restaurants, users, ingredients models and first migration

What: Turned the data model from DESIGN.md into real, versioned database tables.

How:

- app/models/: Restaurant, User, Ingredient as typed SQLAlchemy 2 models (Mapped / mapped\_column)
- role and unit stored as String + CHECK, not a Postgres ENUM, to stay easy to extend
- Composite UNIQUE(restaurant\_id, name) on ingredients doubles as its restaurant\_id index
- First Alembic migration, generated, reviewed and fixed by hand before applying

Evidence: pytest: 1 passed. Verified alembic upgrade head -> downgrade base -> upgrade head all reproduce the same schema via psql.

<https://github.com/Arefhg/dispensa-api/pull/8>

## #9 - feat: add ingredients CRUD with tenant scoping and error envelope

What: The first feature a restaurant can actually use end to end: managing its ingredient list.

How:

- app/services/ingredients.py: create/list/get/update/archive, scoped by (id, restaurant\_id) in one query so another restaurant's id is a plain 404
- app/errors.py: one {"error": {"code", "message"}} shape for every error path, reused by every future router
- tests/conftest.py: real alembic upgrade head against a dispensa\_test database, each test wrapped in a rolled-back transaction
- tests/test\_ingredients.py: 10 tests, including restaurant B's ingredient returning 404 to restaurant A

Evidence: pytest: 10 passed. pre-commit run --all-files clean. Manual curl smoke test for every endpoint and error case.

<https://github.com/Arefhg/dispensa-api/pull/9>

## #10 - docs: add WORKLOG.md and a session-end rule

What: Created a durable record of decisions and lessons, separate from commit messages, that this report itself reads from.

How:

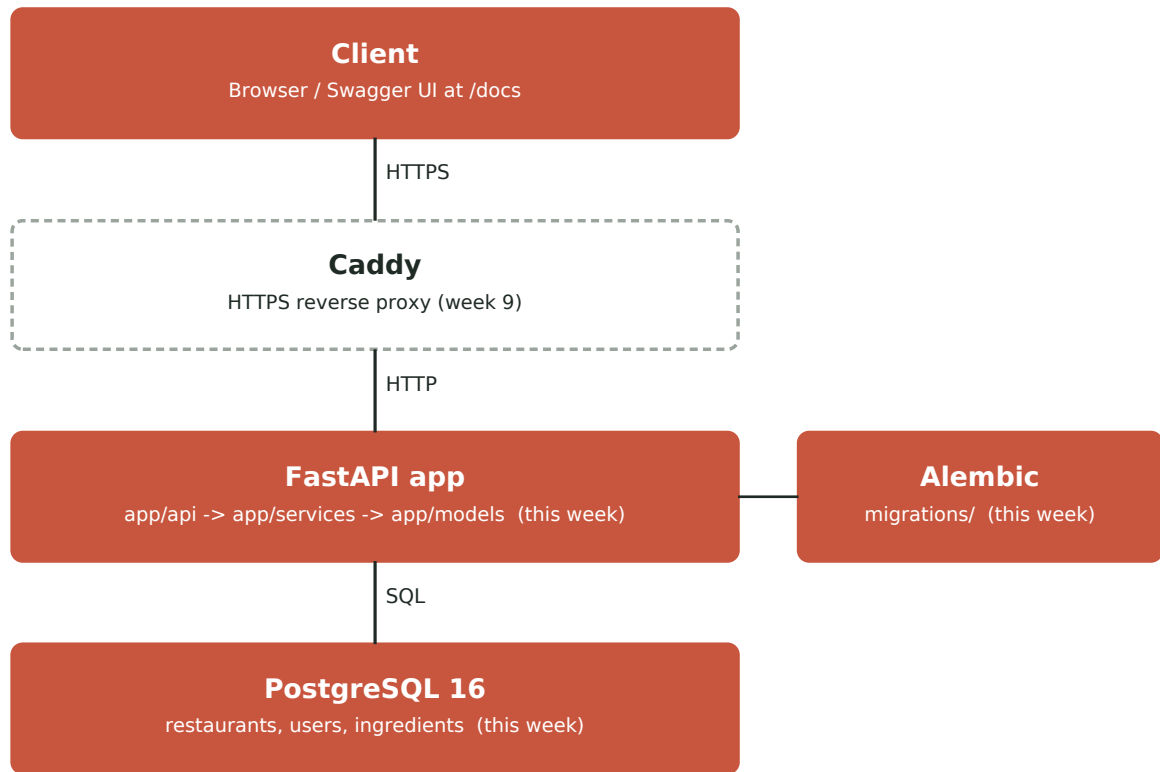
- docs/WORKLOG.md: one entry per session (built, decisions, learned, problems solved, time spent)
- CLAUDE.md: added a session-end rule to keep it updated going forward
- Backfilled three entries from git history and ROADMAP.md

Evidence: Documentation change; reviewed by hand against git history for accuracy.

<https://github.com/Arefhg/dispensa-api/pull/10>

# Architecture snapshot

Dispensa's three-layer architecture as designed in docs/DESIGN.md §2. Solid accent boxes are running today; dashed grey boxes are designed but not yet built.



All containers run on one VPS (from roadmap week 9). Today everything runs locally via Docker Compose + a local venv.

## Technical decisions

From docs/DESIGN.md §10, all made this week.

| Decision                                   | Reason                                                                              | Trade-off                                                      |
|--------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Ledger instead of stored stock             | Every number is explainable; no silent overwrites                                   | Sums on read; needs good indexes                               |
| Recipes & sales moved into the MVP         | Staff's main job is recording sales; without recipes the staff role is almost empty | ~1 extra week; v1.0 target moves to 13 Dec                     |
| Only the owner records deliveries          | Owner handles invoices and prices                                                   | Delivery may be entered after it arrives -> delivered_on field |
| Staff don't see ingredient stock or prices | Owner's choice; staff only need dish availability                                   | Two views of the same data                                     |
| Negative stock allowed                     | Real sales and waste must never be rejected                                         | Owner's stock report can show negative numbers                 |
| Deactivate, never delete users             | History keeps who did what                                                          | Inactive users stay in the table                               |

| Decision                        | Reason                                       | Trade-off                                                          |
|---------------------------------|----------------------------------------------|--------------------------------------------------------------------|
| Shared database + restaurant_id | Simple to run and migrate                    | Isolation depends on every query -> enforced by one helper + tests |
| JWT 8h, no refresh token        | Simple; one login per shift                  | User logs in again after 8 hours                                   |
| PDF/invoice uploads postponed   | Needs storage, limits and backups            | Invoices kept outside the app for now                              |
| Sync SQLAlchemy (not async)     | Simpler code and tests, enough for our scale | Switch to async only if measured load needs it                     |

## Risks and issues

| Risk                                                       | Impact                                                                                       | Mitigation                                                                              | Owner |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------|
| ROADMAP.md checkboxes not ticked as work lands             | Project looks less advanced on paper than it is; makes future status reports harder to trust | Tick checkboxes as part of each PR's definition of done, starting next session          | Aref  |
| No CI pipeline yet - tests only run by hand before merging | A regression could land without being caught automatically until roadmap week 5              | pre-commit + a manual pytest run before every merge until GitHub Actions lands (week 5) | Aref  |
| Week-1 GitHub profile cleanup (issue #1) still open        | Cosmetic only; does not block engineering work                                               | Carry over to next week, low priority                                                   | Aref  |

## Problems hit this week and how they were solved

- localhost resolved to IPv6 on Windows, adding ~90s to every database command → used 127.0.0.1 explicitly everywhere.
- ruff format --check kept failing right after running ruff format → an earlier command in the same shell chain had failed silently; fixed the chaining, not the formatter.
- mypy --strict flagged Settings() as missing an argument it actually gets from the environment at runtime → enabled the pydantic.mypy plugin instead of suppressing the error.
- Two PRs were briefly believed to be merged when they weren't → caught by checking GitHub directly instead of trusting the first report, before any branch was deleted.

## Next week

Roadmap weeks 1-3 are functionally complete, so next week's priorities pull forward from week 5 (tests & CI) rather than repeating already-built week-2/3 items.

| Deliverable                                                                    | Goal                                                                         | Expected outcome                                                        |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 1. Tick ROADMAP.md checkboxes for weeks 1-3                                    | Make the plan match reality before it drifts further                         | Roadmap accurately reflects what's shipped                              |
| 2. GitHub profile cleanup (photo, bio, README) - issue #1                      | Close out the one open week-1 item                                           | Issue #1 closed                                                         |
| 3. Suppliers CRUD with tests                                                   | Next data entity staff/owner need before deliveries can reference a supplier | Suppliers endpoints merged, covered by tests including tenant isolation |
| 4. GitHub Actions: lint + type-check + tests on every push, CI badge in README | Catch regressions automatically instead of relying on manual pytest runs     | A broken test turns the pull request red                                |
| 5. Test coverage report (80%+ target) with badge                               | Make untested code visible instead of assumed                                | Coverage badge in README, baseline measured                             |

## Support needed

None this week.

## Appendix

### Commands and tools used

- git log, git show - history and verifying what actually merged
- gh / GitHub PR list and diffs - pulling PR descriptions and merge status for this report
- pytest - running the 11-test suite against a real PostgreSQL 16 database
- alembic upgrade head / downgrade base - applying and reverting the first migration
- pre-commit run --all-files - ruff check, ruff format, mypy
- docker compose up - running PostgreSQL locally
- psql - inspecting tables, constraints and data by hand

### Glossary

|               |                                                                                                     |
|---------------|-----------------------------------------------------------------------------------------------------|
| API           | A set of URLs a program exposes so other programs can read or change its data.                      |
| CRUD          | The four basic actions on data: Create, Read, Update, Delete.                                       |
| Ledger        | A record where nothing is ever edited, only added to - like a bank statement.                       |
| Migration     | A recorded, reversible change to the database's structure, stored like code.                        |
| Multi-tenancy | One system shared by many customers (restaurants), each seeing only its own data.                   |
| ORM           | Object-relational mapper - lets Python code read/write database rows as typed objects (SQLAlchemy). |

|                   |                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------|
| Pre-commit hook   | A check that runs automatically before a commit is allowed to be created.                    |
| Pull request (PR) | A proposed, reviewed change to the code, shown as a diff before it's merged.                 |
| Tenant scoping    | Filtering every database query by restaurant_id so one restaurant never sees another's data. |